

Sistema di Monitoraggio e Gestione del Traffico Ferroviario

1. Introduzione e Obiettivi

Il progetto mira a realizzare un'architettura distribuita per il monitoraggio in tempo reale della rete ferroviaria. Il sistema deve gestire il flusso di dati proveniente da sensori dislocati nelle stazioni ed eventualmente lungo la rete ferroviaria, elaborare le informazioni centralmente per fornire ad amministratori e dipendenti della centrale operativa una visione d'insieme del traffico e garantire la sicurezza operativa.

Gli obiettivi principali sono:

- **Scalabilità:** Gestire un numero elevato di stazioni e sensori.
 - **Affidabilità:** Garantire che i dati di transito non vadano persi.
 - **Bassa Latenza:** Aggiornare la centrale operativa in tempo reale.
-

2. Architettura generale del Sistema

Il sistema adotta un'architettura **gerarchica a tre livelli**:

1. **Livello Edge (Stazioni):** Ogni stazione è un nodo locale che gestisce i sensori fisici (Rfid, sensori di pressione, fotocellule,).
 2. **Livello di Messaggistica (Middleware):** Un bus di eventi (es. Mosquitto o RabbitMQ) che fa da ponte per trasportare i dati dalle stazioni alla centrale.
 3. **Livello Centralizzato (OCC - Operations Control Center):** Il cuore logico che aggrega i dati, gestisce il database e l'interfaccia di monitoraggio (server REST).
-

3. Componenti attivi

A. Il Nodo Stazione (Sensor Node)

Ogni stazione agisce come un produttore di dati (*Publisher*).

- **Funzionalità:** Rileva l'ingresso in stazione di un treno, l'uscita di un treno e il numero del convoglio che ha attivato i relativi sensori. Un treno può uscire da una stazione senza entrarvi (stazione di partenza del viaggio), oppure entrare senza uscirvi (capolinea).
- **Gestione Guasti temporanei (risolvibili localmente):** In caso di disconnessione dalla centrale, la stazione deve implementare un meccanismo di **caching locale** per reinviare i dati una volta ripristinata la connessione (oppure utilizzare il meccanismo di recapito affidabile offerto dal protocollo MQTT).
- **In caso di guasto non temporaneo** (simulazione della rottura di un binario, o di un treno deragliato che inibisce l'uso di una certa stazione, oppure di una particolare linea della stazione, se volete gestire anche le vie multiple nelle stazioni), è possibile che, dalla centrale operativa, un tecnico possa inviare operatori per risolvere immediatamente il guasto. Il guasto permanente va segnalato immediatamente alla centrale operativa, che quindi potrà attivare la procedura di ripristino inviando gli operatori. Durante la durata del guasto, la

stazione segnala il guasto a ogni treno che entra in stazione, il quale non riparte fino a quando il guasto non è ripristinato.

- **Guasto del Sensore/Stazione (Fail-Stop):**
 - Se una stazione smette di inviare dati alla Centrale Operativa, quest'ultima non capirebbe se la stazione sia guasta o se i treni non transitano, o i sensori non agganciano i treni in arrivo/partenza.
 - Se un sensore smette di inviare dati alla Stazione, quest'ultima non capirebbe più l'andamento del traffico al suo interno e non potrebbe gestire i treni di passaggio.
 - **Rilevamento:** Si suggerisce un meccanismo di **Heartbeat/Keepalive**, sia tra Centrale e Stazioni, sia tra Stazioni e relativi sensori. Se la centrale non ricevesse i "Keep Alive" da una stazione, potrebbe considerarla guasta per qualche motivo; se una Stazione non ricevesse i "Keep Alive" da uno o più sensori, segnalerebbe lo stato di "guasta" alla Centrale Operativa.

B. I treni

Ogni treno è un processo a sè stante (micro-servizio indipendente) che simula il suo viaggio attraverso le stazioni.

- **Funzionalità:** Il treno gestisce il suo presentandosi ai sensori che di volta in volta intercettano la posizione del treno. Il treno è anche dotato di un sistema di comunicazione in ingresso, in grado di ricevere informazioni (guasti o ritardi) dalle stazioni o dalla centrale operativa.

C. La Centrale Operativa (OCC)

Un cluster di server (oppure un server monolitico), che si occupa di:

- **Ingestion:** Ricezione dei flussi di dati dai nodi stazione (tramite protocollo MQTT).
- **Processing:** Calcolo dei ritardi e previsioni di arrivo.
- **Storage:** Database (es. MongoDB o PostgreSQL) per memorizzare lo stato attuale della linea (aggiornandolo con le informazioni che arrivano dal modulo di ingestion, compresi i guasti alle linee) e gestire lo storico dei transiti. La Centrale Operativa memorizza anche le tratte dei treni da gestire, memorizzando il viaggio che ogni treno (indicato dal suo numero di convoglio) deve fare (elenco di stazioni) sia all'andata, che al ritorno.
- **Alerting:** Generazione automatica di allarmi in caso di anomalie (es. treno fermo tra due stazioni per troppo tempo).

Questo server deve essere accessibile da due tipologie di utenti:

- **Tecnico:** visualizza la situazione corrente del traffico e le tratte (ad esempio quali tratte segue un certo treno, oppure quali treni ci si aspetta in ingresso e in uscita in una certa stazione) e ha il compito di mandare operatori per riparare una stazione attualmente guasta.
- **Amministratore:** visualizza la situazione corrente del traffico e le tratte, può aggiungere ed eliminare tratte, può modificare le stazioni attraversate dai treni (modificando quindi le tratte) e (eventualmente) può sopprimere treni fermi in stazione.

4. Protocolli di Comunicazione

Per garantire l'efficienza, si consigliano (come base) i seguenti approcci:

- **MQTT (Message Queuing Telemetry Transport):** Ideale per i sensori delle stazioni grazie al basso overhead e al supporto per reti instabili, e per la comunicazione dei dati dall'edge (stazione/dispositivi/treni) al core (centrale operativa)
- **HTTP:** per la gestione delle API REST offerte dalla centrale operativa agli utenti web del servizio di amministrazione

Simulazione Dinamica

Per rendere il progetto realistico (E OTTENERE LA SUFFICIENZA), la simulazione non deve essere un unico blocco di codice, ma un **ecosistema di agenti**.

A. I "Digital Twins" dei Treni

Ogni treno può essere modellato come un processo separato (o un thread isolato) che mantiene il proprio stato:

- **Stato Interno:** ID treno (numero di convoglio), stazione precedente, prossima stazione. La prossima stazione da raggiungere può essere recuperata da una struttura del viaggio pre-caricata nella sua memoria alla partenza (da quale stazione parte, in quali stazioni fermarsi e terminare in quale stazione capolinea), oppure indicata via via dalla stazione nella quale il treno si ferma: ad esempio
 - la stazione chiede alla centrale operativa la prossima stazione dato <ID Treno, stazione attuale, (A)ndata o (R)itorno>
 - la Centrale recupera la prossima stazione dal suo DB e la invia alla stazione come risposta;
 - la stazione manda un messaggio al treno indicandogli la prossima stazione da raggiungere
- **Comportamento:** Il treno invia "ping" GPS o segnali di prossimità ai sensori delle stazioni tramite protocolli leggeri come **MQTT**.
- Per muoversi, il treno può implementare un motore basato su **eventi discreti**:
 1. Il motore conosce il tempo di percorrenza tra stazione attuale e prossima stazione. Questo tempo può essere preso dalla tabella pre-caricata in memoria, o gli viene comunicato dalla stazione, insieme alla prossima stazione, se avete scelto quest'ultima opzione.
 2. Allo scadere del tempo (considerando variabili come rallentamenti o traffico), invia un trigger al "Sensore di Stazione in ingresso" della prossima stazione..
 3. Il Sensore, a sua volta, avverte la stazione, la quale inoltra il pacchetto dati alla Centrale Operativa, che aggiorna la situazione del viaggio del treno in oggetto.

Schema a Blocchi dell'Architettura

Il sistema si articola su quattro livelli principali:

A. Livello Edge (Produzione Dati)

È il fronte fisico (o simulato) dove avviene il rilevamento.

- **Train Agents (Simulatori):** Processi indipendenti che simulano il movimento. Inviando segnali di posizione.
- **Station Nodes (+ Sensori):** Microservizi che filtrano i dati dei treni.
 - *Componenti interni:* Logic Controller, Local Buffer (per guasti rete), Heartbeat Generator, ...

B. Livello di Messaggistica (Disaccoppiamento)

È il "sistema nervoso" che permette la comunicazione asincrona.

- **Message Broker (es. Mosquitto/RabbitMQ):** Gestisce i canali di comunicazione con il campo (Topic). Ad esempio:
 - `topic/train/telemetry`: Dati sulla posizione.
 - `topic/station/status`: Heartbeat e diagnostica stazioni.
 - `topic/alerts`: Comandi di emergenza dalla centrale.

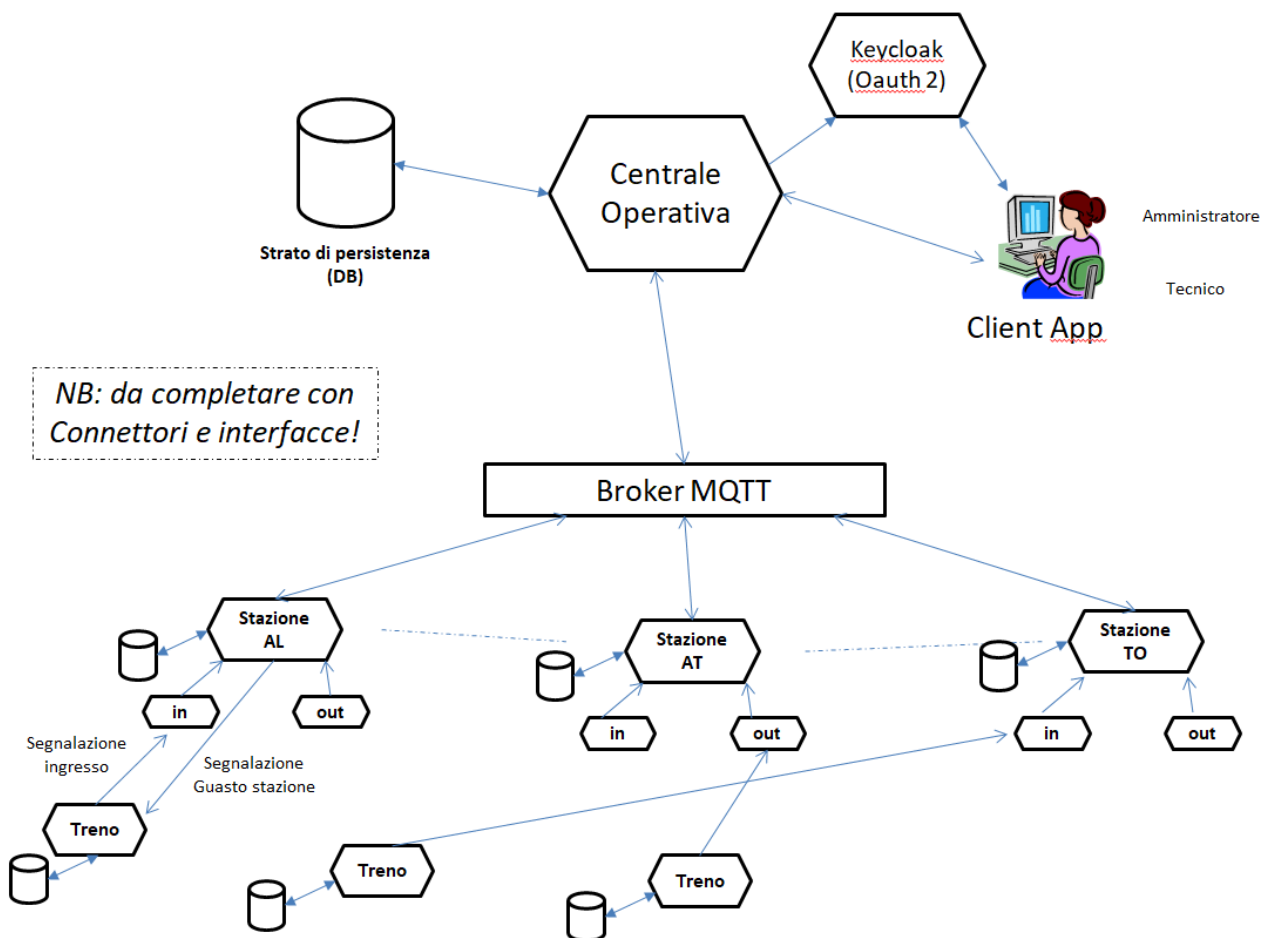
C. Livello Core (Centrale Operativa - OCC)

Il cervello del sistema, progettato per l'alta affidabilità. Può essere monolitico o spezzato in servizi indipendenti, ad esempio:

- **Ingestion Service:** Riceve i messaggi dal broker e li valida.
- **Traffic Logic Engine:** Restituisce le tratte e la prossima stazione per treni in transito.
- **Fault Manager:** Monitora gli heartbeat delle stazioni; se una stazione "muore", attiva eventuali procedure di emergenza.
- ...

D. Livello di Persistenza e Dashboard

- **Data Base (Postgres, MySQL, ...):** Per memorizzare la mappa della ferrovia, i passaggi, i treni in movimento, quelli fermi in stazione e così via.
- **Interfaccia Web:** consultabile da amministratori e tecnici.



Questa è solo un'idea di massima dell'architettura generale, che potete rivedere a piacere a seconda delle vostre scelte progettuali. Ad esempio, stazione, sensori e treno non comunicano "direttamente", ma tramite "Topic" MQTT e possono essere, per semplicità, tutti "fisicamente" connessi allo stesso broker ma fanno arrivare i messaggi corretti ai destinatari corretti proprio grazie ai topic che voi dovreste identificare e documentare.

Ogni componente (stazione, Centrale Operativa), può essere a sua volta scomposta in micro-servizi più semplici da gestire, che "parlano" tra di loro per coordinarsi. Nella relazione finale, ricordate di completare gli esagoni con le opportune interfacce e di documentare i protocolli di comunicazione tramite i relativi diagrammi di sequenza.

Varianti Ammesse

Rispetto alla Architettura base in Figura si ammettono varianti / semplificazioni, alcune delle quali non comportano penalità per quanto concerne la valutazione, mentre altre pongono un limite al punteggio massimo ottenibile.

- Variante 1: Si può scegliere se implementare una SPA o una WEB-APP (nessuna penalizzazione).
- Variante 2: invece di usare OAUT2, si usa un sistema di autenticazione dove gli utenti sono memorizzati nel Server e l'autenticazione avviene col metodo tradizionale (login/Passwd) (nessuna penalizzazione).
- Variante 3: I DB possono essere implementati con un vero DBMS accessibile tramite JDBC, oppure tramite files Json serializzati sul file system (nessuna penalizzazione)
- Variante 4: Invece di comunicare via MQTT, le stazioni comunicano usando API REST eventualmente offerte dal management service stesso (5 punti di penalizzazione).
- Variante 5: invece di sviluppare la SPA / WEB-APP, si simula l'interazione con l'utente del management service usando una utility come Postman o altro (penalità 2 punti).
- Variante 6: non si implementa la comunicazione su TLS (penalità 5 punti).
- Variante 7: i servizi di monitoraggio delle stazioni sono tutti raggruppati in un unico servizio (penalità 5 punti).

Più di una variante può essere scelta. In caso di varianti penalizzate, le penalità verranno sommate.

Valutazione

Il progetto deve essere consegnato almeno **UNA SETTIMANA** prima della data d'esame, che sarà concordata con il docente. La consegna prevede:

1. Il codice sviluppato
2. Un rapporto tecnico steso secondo le linee guida pubblicate sulla pagina del corso

All'atto dell'interrogazione lo studente dovrà dimostrare il funzionamento del progetto sviluppato.

Sia il codice, che il rapporto riceveranno una valutazione che concorrerà all'assegnamento del punteggio finale del progetto.

CAVEAT:

Poichè il progetto è uguale per tutti e risulta quindi molto facile avvalersi di contributi di lavoro sviluppato da altri, si rammenta che:

- Il docente userà strumenti antiplagio per rilevare se il codice presentato contenga parti sostanziali di codice presente in progetti presentati da altri studenti.
- All'atto della interrogazione il docente si riserva di chiedere spiegazioni dettagliate del funzionamento di qualunque parte del codice presentato.
- Chi non sapesse dare spiegazioni esaurienti relativamente a parti significative di codice verrà respinto!